

UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA



Curso:

Base de Datos II

Laboratorio 04.5: Indexación con Árbol B+ en Memoria Secundaria

Alumno:

Llerena Silva, Nicolás Alejandro (202110190)

Profesor:

Lovon, Percy

Lima, setiembre de 2026

Introducción

El laboratorio pide construir un índice de árbol B+ que viva en disco y evaluarlo sobre el dataset de empleados, usando el `Employee_ID` como clave. La diferencia con las estructuras de los laboratorios anteriores es el grano: acá la unidad de trabajo no es el registro sino el bloque, y cada nodo del árbol ocupa exactamente un bloque del archivo. Esa decisión es la que hace que el árbol sea plano y que una búsqueda cueste tres lecturas incluso con cien mil registros.

Están implementadas las cuatro operaciones que pide el enunciado (insert con split, search, rangeSearch aprovechando el enlace entre hojas y remove con rebalanceo real), más un validador de invariantes que se usa como red de seguridad en todas las pruebas. Todo el código es Python con struct y archivos binarios; matplotlib solo aparece para las gráficas.

Los datos y el registro

El `employee.csv` se construye desde la base `employees` del Laboratorio 01 con `exportar_csv.py`, que toma 100 000 empleados vigentes con sus datos reales de nombre, departamento, sueldo y fecha de ingreso. `Country` es el único campo que la base no tiene y se asigna de forma determinista a partir del id. Los 100 000 registros permiten llegar al $N = 10^5$ que sugiere el enunciado para los experimentos.

Sobre el tamaño del registro hay que hacer una precisión. El enunciado anuncia 112 bytes, pero la tabla de campos que da a continuación lista seis campos que suman 88: 4 de `Employee_ID`, 30 de `Employee_Name`, 20 de `Country`, 20 de `Department`, 4 de `Salary` y 10 de `Joining_Date`. Los 112 corresponden al registro del laboratorio anterior, que además tenía `Age` y `Position`. Implementé lo que dice la tabla, con los formatos de struct que ella misma indica, así que el registro ocupa 88 bytes y el formato completo es `<i30s20s20sf10s`.

```

bash — labs/4.5

$ python3 exportar_csv.py 100000
data/employee.csv: 100000 registros, 5780165 bytes
Employee_ID,Employee_Name,Country,Department,Salary,Joining_Date
10001,Georgi Facello,Brasil,Development,95000,26/06/1986
10002,Bezalel Simmel,Espana,Sales,88000,21/11/1985
10003,Parto Bamford,Uruguay,Production,43311,28/08/1986
$

```

Exportación del CSV desde la base employees.

P1: estructura física y serialización

Un nodo es un bloque y un bloque es un nodo. El tamaño del bloque no se fija a mano sino que se deriva del orden M , porque lo que tiene que entrar en el bloque son $M-1$ registros de 88 bytes más la cabecera: $BLOQUE(M) = 12 + (M-1) \times 88$. Con el orden 64 que uso por defecto eso da bloques de 5556 bytes. El nodo interno entra holgado en ese mismo bloque, porque sus $M-1$ claves de 4 bytes y sus M punteros de 4 bytes ocupan mucho menos que los registros de una hoja; la diferencia queda como relleno, que es el precio de tener un único tamaño de bloque para los dos tipos de nodo.

Cabecera del bloque (12 bytes)

tipo 1 B	reservado 1 B	n_claves 2 B	next_leaf 4 B	prev_leaf 4 B
-------------	------------------	-----------------	------------------	------------------

Nodo interno (bloque de 5556 bytes)

cabecera 12 B	claves separadoras x 63 252 B (4 B c/u)	punteros a bloque x 64 256 B (4 B c/u)	relleno 5036 B
------------------	--	---	-------------------

Nodo hoja (bloque de 5556 bytes)

cabecera 12 B	registros completos x 63 5544 B (88 B c/u: id, nombre, país, área, sueldo, fecha)
------------------	--

next_leaf y prev_leaf solo se usan en las hojas; en los nodos internos valen -1.

Estructura binaria de los bloques en disco, con el orden $M = 64$.

La cabecera de 12 bytes lleva el tipo de nodo, un byte reservado para alinear, el número de claves ocupadas y los dos punteros de la lista de hojas. En un nodo interno esos dos punteros valen -1. Después de la cabecera, un nodo interno guarda sus claves seguidas de sus punteros a bloque, y una hoja

guarda los registros completos uno detrás de otro. Los punteros son números de bloque, no desplazamientos en bytes, así que el offset físico se calcula como cabecera del archivo más número de bloque por tamaño de bloque.

`read_node` hace un seek al bloque, lee sus bytes de una vez y los interpreta con `struct.unpack_from` según el tipo; `write_node` arma un bytearray del tamaño exacto del bloque, lo rellena con `struct.pack_into` y lo escribe de un solo write. Las dos funciones incrementan los contadores de lecturas y escrituras, que son los que alimentan todas las mediciones de la tercera parte. El archivo empieza con una cabecera propia de 24 bytes que guarda la raíz, la cantidad de bloques, la altura, el número de registros, el orden y la cabeza de la lista de bloques libres.

La inserción baja hasta la hoja que corresponde, mete el registro en su posición ordenada y, si la hoja se pasa de $M-1$, la parte en dos y devuelve hacia arriba la primera clave de la mitad derecha. En una hoja esa clave se copia al padre y en un nodo interno la clave del medio sube y desaparece del hijo, que es la diferencia clásica entre el split de hoja y el de nodo interno en un B+. Si la promoción llega hasta la raíz se crea una raíz nueva y el árbol gana un nivel. Con 100 000 registros el árbol terminó con altura 3 y 2296 hojas.

P2: búsquedas y eliminación

`search` desciende de la raíz a la hoja resolviendo cada nodo interno con búsqueda binaria sobre sus claves separadoras, así que hace exactamente una lectura por nivel. `rangeSearch` hace ese mismo descenso una sola vez, para ubicar la hoja donde empieza el rango, y a partir de ahí avanza por el puntero `next_leaf` leyendo hojas consecutivas hasta pasarse de la clave final. Es la operación donde el B+ se separa del resto: los niveles superiores del árbol no se vuelven a tocar.

`remove` elimina físicamente y rebalancea. Se quita el registro de la hoja y, si queda por debajo del mínimo, se intenta primero pedirle un registro al hermano izquierdo y después al derecho, actualizando la clave separadora del padre. Si ningún hermano puede prestar sin quedar corto, los dos se fusionan en uno y la clave separadora baja o desaparece según el nodo sea interno u hoja. La fusión puede dejar corto al padre, y eso lo resuelve la vuelta de la recursión; si

la raíz se queda sin claves, el árbol pierde un nivel. Los bloques que quedan vacíos por una fusión se encadenan en una lista de libres y se reutilizan en la siguiente inserción, para que el archivo no crezca indefinidamente.

Como la eliminación es la parte donde estos árboles se rompen en silencio, el módulo trae un validador que recorre el árbol entero y comprueba cinco cosas: que las claves de cada nodo estén ordenadas, que caigan dentro del rango que impone el padre, que todas las hojas estén a la misma profundidad, que ningún nodo distinto de la raíz esté por debajo del mínimo de ocupación y que el recorrido por la lista de hojas devuelva exactamente los registros esperados en orden. La prueba que aparece abajo inserta 3000 registros y borra 2500 con tres órdenes distintos, validando cada 500 bajas.

```
bash - Labs/4.5

$ python3 bplus_tree.py
B+ Tree File sobre data/employee.csv
orden M      : 64 (hasta 63 claves por nodo)
tamano de bloque : 5556 bytes = 12 de cabecera + 63 x 88
registros      : 5000
altura        : 3
bloques usados  : 161 (894540 bytes)
search(13092) -> Kersti Bergere, Marketing [3 lecturas]
search(1) -> None
insert(999999) -> 3 lecturas, 1 escrituras, altura=3
search(999999) -> Nicolas Llerena
rangeSearch(10013, 10513) -> 402 registros [15 lecturas]
    10013 Eberhardt Terkki      Human Resources
    10014 Berni Genin          Development
    10016 Kazuhito Cappelletti Sales
remove(13092) -> True [11 lecturas, 5 escrituras]
search(13092) tras eliminar -> None
validacion      : 5000 registros, altura 3, 155 hojas, ocupacion 51%
prueba de estres de la eliminacion (invariantes tras cada tanda):
    M= 4: 3000 altas y 2500 bajas -> quedan 500 registros, altura 7, 222 hojas, invariantes correctas
    M= 8: 3000 altas y 2500 bajas -> quedan 500 registros, altura 4, 102 hojas, invariantes correctas
    M= 64: 3000 altas y 2500 bajas -> quedan 500 registros, altura 2, 13 hojas, invariantes correctas

$
```

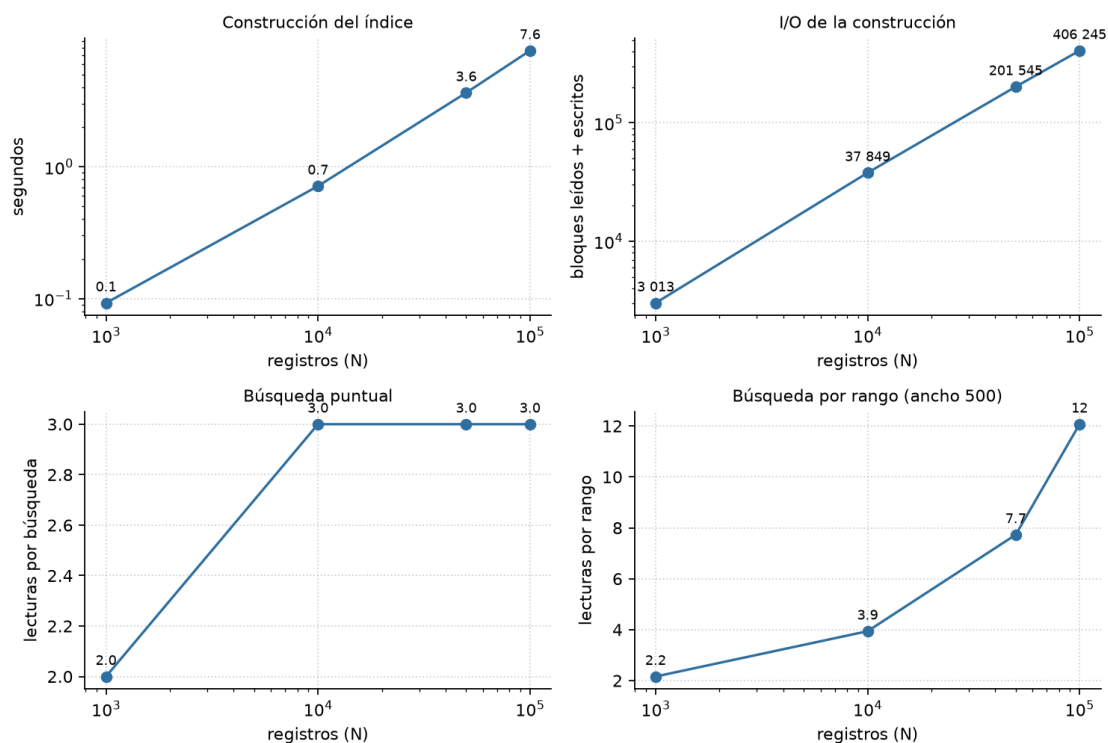
Operaciones sobre 5000 registros y prueba de estrés de la eliminación. Entre corchetes van los accesos a disco.

P3: evaluación experimental

Las mediciones se hacen con benchmark.py sobre el CSV barajado con una semilla fija, para que las claves no entren ordenadas y el árbol se comporte como en un caso real. Cada tanda cronometra la operación y lee los contadores de bloques leídos y escritos que llevan read_node y write_node. Después de construir y después de eliminar se corre el validador, así que los números que siguen corresponden a árboles verificados.

N	Altura	Hojas	Ocupación	Construcción		Búsqueda		Rango	
				seg	I/O	lecturas	ms	lecturas	ms
1 000	2	23	69%	0,09	3 013	2.0	0,041	2.2	0,043
10 000	3	222	72%	0,71	37 849	3.0	0,044	3.9	0,086
50 000	3	1 135	70%	3,64	201 545	3.0	0,043	7.7	0,215
100 000	3	2 296	69%	7,60	406 245	3.0	0,042	12.1	0,360

Escalamiento con $M = 64$. La búsqueda y el rango son promedios de 500 y 100 operaciones.



Costo de construcción, búsqueda y rango al crecer N .

El resultado que más importa está en el panel de abajo a la izquierda: la búsqueda puntual cuesta 2 lecturas con mil registros y 3 con cien mil, y esas lecturas son exactamente la altura del árbol. Entre 10^4 y 10^5 el número no se mueve porque un árbol de altura 3 con orden 64 admite hasta $64 \times 64 \times 63 = 258\,048$ registros; para pasar a cuatro lecturas habría que superar ese techo. La construcción, en cambio, crece de forma lineal, que es lo esperable: son N inserciones y cada una cuesta del orden de la altura.

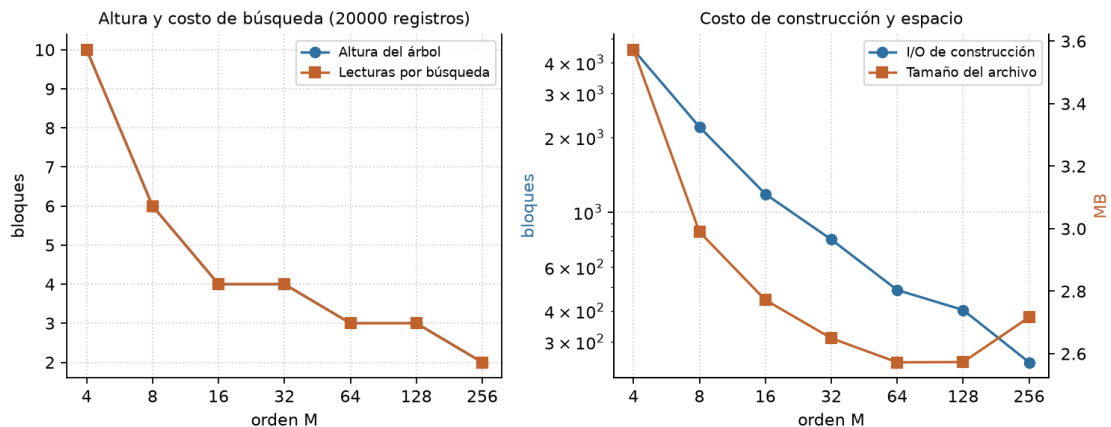
El costo del rango crece de 2.2 a 12.1 lecturas, pero no porque el árbol empeore: el rango tiene ancho fijo de 500 claves y, al haber más registros en el mismo tramo de identificadores, la respuesta trae más filas y hacen falta más

hojas para devolverlas. El descenso sigue costando 3 lecturas; el resto es recorrido secuencial por `next_leaf`.

Impacto del orden M

M	Bloque (B)	Altura	Hojas	Archivo (MB)	Búsqueda (lecturas)	Rango (lecturas)	I/O construcción
4	276	10	8 546	3,57	10.0	45.1	4 508
8	628	6	3 940	2,99	6.0	22.0	2 202
16	1 332	4	1 897	2,77	4.0	11.8	1 182
32	2 740	4	925	2,65	4.0	7.8	775
64	5 556	3	454	2,57	3.0	4.8	485
128	11 188	3	227	2,57	3.0	4.0	403
256	22 452	2	120	2,72	2.0	2.5	248

Efecto del orden con 20 000 registros.



Altura y costo de búsqueda frente al orden, y precio de un M grande.

Subir el orden aplana el árbol y eso se traduce directamente en menos lecturas: con $M = 4$ el árbol tiene altura 10 y cada búsqueda cuesta 10 lecturas, mientras que con $M = 256$ bastan 2. La ganancia no es gratis y se nota en dos lugares. El primero es el bloque: con $M = 256$ pesa 22 452 bytes, de modo que cada lectura mueve 22 KB para responder por un registro de 88 bytes, y el sistema de archivos termina haciendo varias operaciones físicas por cada lectura lógica. El segundo es la escritura: cada inserción reescribe el bloque entero, así que un bloque grande encarece la construcción. En un motor real M se elige para que el bloque coincida con la página del disco, típicamente 4 u 8 KB, que en este diseño cae entre $M = 32$ y $M = 64$.

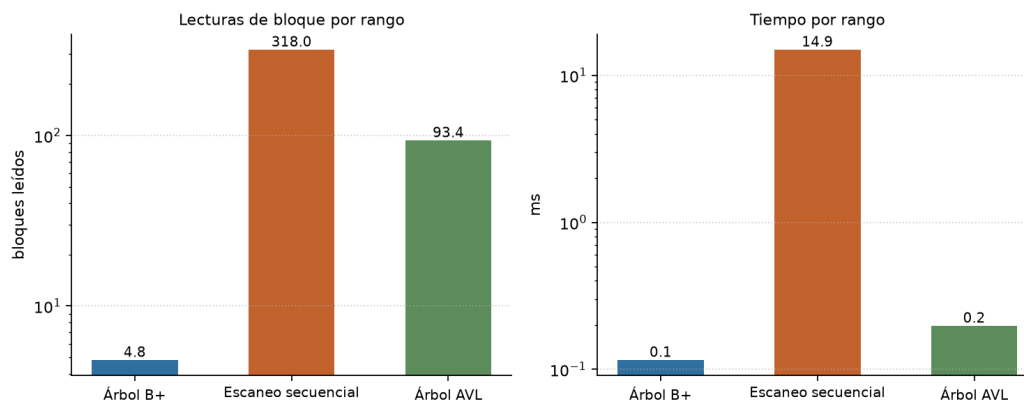
El tamaño del archivo se comporta al revés de lo que uno esperaría: baja de 3,57 MB con $M = 4$ a 2,57 MB con $M = 64$. Con órdenes chicos hay muchísimos

nodos internos, y cada uno gasta una cabecera y punteros; además el relleno del bloque interno se paga tantas veces como nodos internos haya. Con $M = 256$ vuelve a subir un poco porque las hojas quedan medio vacías y cada hoja desperdicia media página.

Rango: B+ contra escaneo secuencial y contra el AVL

Estructura	Lecturas por rango	Tiempo por rango (ms)	Veces más lecturas
Árbol B+ ($M = 64$)	4.8	0,115	1,0
Árbol AVL (laboratorio 04)	93.4	0,197	19,4
Escaneo secuencial	318.0	14,902	66,1

Rangos de 500 claves sobre 20 000 registros, que devuelven 80 registros en promedio. Los tres métodos devuelven exactamente los mismos registros, lo verifica el benchmark.



Costo de una búsqueda por rango según la organización del archivo.

El escaneo secuencial lee el archivo completo, 318 bloques, para devolver 80 registros; tarda 14,9 ms, unas 129 veces más que el B+. El AVL es mucho mejor que eso porque no lee lo que no necesita, pero igual gasta 93 lecturas contra las 4.8 del B+: como cada nodo del AVL guarda un solo registro, devolver 80 filas exige visitar al menos 80 nodos, más el camino de bajada. El B+ resuelve lo mismo con tres lecturas de descenso y menos de dos hojas.

Por qué el B+ minimiza los accesos a disco

La razón de fondo es que en memoria secundaria el costo se paga por bloque y no por comparación. Traer un bloque cuesta un posicionamiento del disco, y ese posicionamiento es órdenes de magnitud más caro que cualquier trabajo que se haga después en RAM con los bytes traídos. Un árbol binario gasta ese

acceso completo para leer un solo registro y dos punteros; el B+ lo gasta para traer hasta 63 claves de una vez y decidir con ellas por dónde seguir.

Eso se ve en la altura, que es lo único que se paga en una búsqueda. Un AVL tiene altura de hasta $1,44 \cdot \log_2(N)$, que con 100 000 registros son unos 24 niveles; el B+ tiene altura $\log_M(N)$, que con $M = 64$ son 3. Medido sobre 20 000 registros, la construcción del AVL del laboratorio anterior gastó 527 000 accesos y la del B+ unos 38 000 para 10 000 registros, es decir un orden de magnitud menos por registro insertado.

A eso se suma la segunda decisión de diseño del B+: los registros viven solo en las hojas y las hojas están enlazadas. Un árbol binario mezcla datos y estructura en todos los niveles, así que un recorrido ordenado obliga a subir y bajar por el árbol saltando entre bloques dispersos. En el B+ el recorrido ordenado es leer hojas consecutivas, que además es el patrón que mejor aprovecha la lectura anticipada del sistema operativo. Por eso el B+ es la estructura que usan PostgreSQL, MySQL y prácticamente cualquier motor para sus índices, y no un AVL.

Conclusiones

El árbol B+ quedó implementado con bloques de tamaño fijo, un nodo por bloque, split en la inserción, eliminación con préstamo y fusión, y lista enlazada entre hojas. Con 100 000 registros el árbol tiene altura 3 y una búsqueda cuesta 3 lecturas, un número que se mantiene constante en dos órdenes de magnitud de N . Es la propiedad que justifica la estructura: el costo de acceso deja de depender del tamaño de los datos en cualquier rango de tamaños práctico.

En la búsqueda por rango la ventaja es todavía más marcada, porque el enlace entre hojas convierte la consulta en una lectura secuencial: 4.8 bloques contra 93 del AVL y 318 del escaneo completo. El orden M es la perilla que gobierna todo el comportamiento, y conviene elegirlo de modo que el bloque coincida con la página física del disco: más chico deja el árbol innecesariamente alto y más grande hace que cada acceso mueva datos que no se usan.

Anexo: archivos entregados

registro.py define el registro de 88 bytes, su serialización con struct y la lectura del CSV. bplus_tree.py contiene la clase BPlusTreeFile con read_node, write_node, insert con split, search, range_search, remove con préstamo y fusión, y el validador de invariantes; ejecutado directamente corre la demostración y la prueba de estrés de este informe. avl_file.py es el árbol AVL del laboratorio anterior, incluido porque benchmark.py lo usa como línea base de la comparación por rango. benchmark.py ejecuta los tres experimentos y genera las gráficas y el diagrama de bloques. Los datos van en data/employee.csv, exportado de la base employees del Laboratorio 01 tal como se describe al inicio del informe.